

CC3104 – Aprendizaje por Refuerzo

Laboratorio 3

Francis Aguilar, 22243

César López, 22535

José Marchena, 22398

Instrucciones

Una empresa de desarrollo de videojuegos está evaluando el uso de agentes de RL para generar comportamiento no determinista en personajes de un juego de rol táctico. El entorno de prueba es un mapa de cuadrícula de **6 × 6** con zonas de recompensa, zonas de penalización y un estado terminal.

El equipo técnico necesita entender si Monte Carlo es viable para este dominio antes de escalar a métodos más complejos, y quiere comparar su comportamiento contra la línea base de **Programación Dinámica** de la semana anterior.

Su grupo ha sido contratado para diseñar el experimento, implementar los métodos, analizar los resultados y producir un reporte técnico con recomendaciones sobre la viabilidad de Monte Carlo para este dominio.

Task 1 (Entrega Parcial)

Antes de implementar nada, diseñen formalmente el experimento que van a ejecutar. El diseño debe incluir:

1. Especificación del MDP:

Definan el espacio de estados, el espacio de acciones, la función de recompensa y el factor de descuento γ para el mapa 6 × 6. Justifiquen cada decisión de diseño. El mapa debe incluir al menos:

- Dos zonas de recompensa positiva.
- Una zona de penalización.
- Un estado terminal.

La función de recompensa debe capturar al menos dos objetivos potencialmente conflictivos propios del dominio de videojuegos.

Mapa propuesto:

Inicio	.	.	.	.	.
.	.	.	.	.	.
.	.	.	.	.	.
.	.	.	Penalización	Objetivo	.
.	.	.	.	.	.
.	Objetivo	.	.	.	Terminal

Estado:

- X, Y: coordenada de la celda donde se encuentra el agente (0-5)
- v_i : Representativo de las celdas de recompensa i y determina si ya fueron visitadas o no, es decir, si la recompensa ya ha sido capturada ($S = (x, y, v1)$) Donde $x, y \in 0, 1, \dots, 5$ $v_i \in 0, 1$

Acciones:

- Movimiento: (arriba, abajo, izquierda, derecha) $A = (\text{arriba}), (\text{abajo}), (\text{izquierda}), (\text{derecha})$

Con desplazamientos $\delta = (0, -1), (0, 1), (-1, 0), (1, 0)$

Transición: $P(s'|s, a)$

Determinsta.

- Bordes: no puede moverse a los bordes y las transiciones imposibles se eliminan.
- Estado terminal: absorbente

Recompensa: $R(s, a, s')$

- El estado terminal debe tener alta recompensa para asegurar que el agente logre llegar a este destino. $+100 \text{ sis}' = \text{terminal}$
- Los estados de recompensa deben sumar menos del estado final, esto asegura que incluso sin recompensa, el modelo logre llegar al final pero que de igual forma busque recompensas. $+5 \text{ sis}' v_i = 0$, es la primera visita
- Penalización debe ser mayor a la de las zonas de recompensa, de esta manera el agente evita estos estados sin entrar en un bucle de recompensa. $-15 \text{ sis}' \in \text{penalización}$
- Cada transición tiene un costo pequeño, pero indeseable para el agente. $-1 \text{ cualquier trocero}$

Esto cumple las restricciones $R_{\text{terminal}} > \sum_i R_{r_i} : 100 > 5 + 5 = 10$ (se llega al final aunque se ignore la recompensa)

$R_{\text{penalización}} > R_{r_i} : 15 > 5$ (evita penalización aunque eso signifique perder una recompensa)

Factor de descuento

$$\gamma = 0.99$$

γ cercano a 1 hace que el agente valore recompensas lejanas casi tanto como inmediatas, necesario cuando el camino óptimo (recolectar ambas recompensas y evitar ambas penalizaciones) requiere planificación a varios pasos.

2. Selección de variante Monte Carlo:

Argumenten cuál variante usarán (First-Visit o Every-Visit) y por qué es más apropiada para este dominio específico. Incluyan en su argumento una discusión sobre la frecuencia esperada de revisitas a estados en un mapa 6×6 bajo una política aleatoria.

Elección: First Visit Para un estado es en el estimador de valor por montecarlo, G_t considera solo la primera vez que aparece s en cada episodio. $V(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i(s)$

Se prefiere esto para nuestro caso, pues nos da independencia de las muestras lo cual nos ayuda a converger más rápido al introducir menos sesgo.

También, dado que tenemos un bajo dominio dde $|S| = 6 \times 6 = 144$ la probabilidad de caer nuevamente en una misma casilla es alta, lo cual inflaría la cantidad de muestras correlacionadas de Every-Visit haciendolo menos deseable.

3. Estrategia de exploración:

Argumenten si usarán Exploring Starts o una política ϵ -soft. Justifiquen su elección considerando si en el dominio de videojuegos es razonable controlar el estado inicial del agente. Propongan un valor concreto de ϵ si eligen política ϵ -soft, con justificación.

Elección: ϵ -soft. Uno de los casos de uso mas importantes para Exploring Starts, es la idea que el inicio no sea importante o que un agente pueda "teletransportarse" entre tuplas (s,a) . En el caso del dominio de videojuegos, esto no es del todo razonable.

Un personaje en un RPG táctico tendrá una posición fija de inicio por cuestiones de diseño de nivel o narrativa. Por lo que forzar inicios arbitrarios, no hace del todo sentido porque podrian haber circunstancias imposibles que se simulen. Aunque esto no descarta ES por completo, igual no es un sistema escalable si se expande el dominio del videojuego.

Por tanto, se opta por una politica ϵ -greedy con caso particular ϵ -soft, que garantiza que $\pi(a|b) \geq \frac{\epsilon}{|A|}$ para toda acción, manteniendo el inicio del episodio fijo y delegando exploracion a la aleatoriedad.

Asimismo se propone un valor de $\epsilon = 0.1$ al ser el estandar que cumple estas condiciones, que nos indica que caa acción suboptima tiene 0.025 minimo de probabilidad de ser tomada.

4. Hipótesis de comparación:

Antes de ejecutar el experimento, formulen al menos dos hipótesis concretas sobre cómo esperan que se comporte Monte Carlo en comparación con Value Iteration de la semana pasada. Las hipótesis deben ser falsables y cuantificables.

- H1: Convergencia Tras un $N = 5000$ episodios de entrenamiento, la política π_{mc} convergerá a un valor promedio $V^{\pi_{MC}}(s_0)$ que se ubicara dentro de 5% del valor óptimo de $V^*(s_0)$ bajo DP. Asimismo, la varianza de corridas independientes con 10 semillas distintas de MC debe ser considerablemente mayor que DP.

Es falsable y cuantificable por el 5%.

- H2: Costo Computacional Monte Carlo First-Visit convergerá al menos una orden de magnitud mas de actualizaciones de estado-valor en comparación con Value Iteration para conseguirun error absoluto promedio menor a $0.05 \times V^*$.

Es falseable y cuantificable por el error absoluto como criterio y la diferencia de orden de magnitud como condicion de cumplimiento.

Task 2 (Entrega Parcial)

Respondan las siguientes preguntas con argumentación técnica.

1. Para el MDP que diseñaron, calculen una cota superior del número de episodios necesarios para que todos los pares (s, a) sean visitados al menos una vez en esperanza, bajo una política uniforme aleatoria. Expresen el resultado en función de $|S|$ y $|A|$ y evalúen numéricamente para su MDP específico.

Con la politica uniformemente aleatoria, en cada paso el agente elige una accion con la misma probabilidad independiente del estado. Esto se puede hacer simil con el problema del coleccionista de cupones, el cual ya tiene una cota superior definida.

$E(\text{pasosnecesarios}) \approx n \ln(n)$ Donde nuestro n es $|S| + |A|$

La unica transofrmacion necesaria, es que no estamos pensando en episodios no pasos, y cada episodio nuestro es una serie de pasos. Podemos realizar una aproximacion con un valor $d_{\min}$ que representaria la longitud minima de un episodio.

Por tanto

$$(N)_{\text{episodios}} \approx \frac{|S||A|\ln(|S||A|)}{d_{\min}}$$

Sustituyendo valores: $|S| = 6 \times 6 \times 2 \times 2 = 144$ (x, y, v1, v2, asimuiendo 2 objetivos) $|A| = 4$ (4 opciones) $n = 144 * 4 = 576$

$d_{\min} = |x_{\text{terminal}} - x_{\text{inicio}}| + |y_{\text{terminal}} - y_{\text{inicio}}|$ Y dado que nuestra grilla esta limitada por las coordenadas (0,0) y (5,5), este valor es 10.

$$(N)_{\text{episodios}} \approx \frac{576 \ln(576)}{10} \approx 370$$

Este valor nos puede servir como guía, pues realmente, el caso real requerirá de más pasos, pues no todos los pares son igual de fáciles que alcanzar.

2. El retorno G_t es un estimador insesgado pero de alta varianza de $V^{\pi}(s)$

Expliquen formalmente:

- De dónde proviene esa varianza.
- Por qué crece con la longitud del episodio.
- Qué consecuencia tiene sobre el número de episodios necesarios para convergencia práctica.

En nuestro caso de First-Visit MC estima su valor con la siguiente fórmula. Esto es insesgado debido a que la función G_t tiene dos fuentes de aleatoriedad que se van apoyando a sí mismo:

- Estocasticidad de la política
- Estocasticidad de la transición Cada elección de acciones puede llegar a un camino distinto, y por tanto, un G_t distinto. La varianza de G_t depende directamente de la varianza de estas variables aleatorias.

Esta varianza crece con la longitud del episodio, sin ahondar en la matemática, porque cada recompensa es aproximadamente independiente en cada paso. Es decir, esta varianza se acumula con cada paso. Asimismo, dado que tomamos un γ alto, este factor también extiende la relevancia de los pasos futuros para el retorno esperado, lo que de igual manera, aumenta mucho más su varianza.

Su consecuencia se puede ver directamente por la siguiente ecuación derivada de la ley de los grandes números con el error de Montecarlo:

$$SE(\hat{V}(s)) = \frac{\sqrt{\text{Var}(G_t|S_t=s)}}{\sqrt{N(s)}}$$

Esta ecuación nos permite ver la relación cuadrada que tiene la varianza con el error estándar, es decir, para reducir el error a la mitad, necesitamos reducir la varianza por un factor de 4. Esto sumado con la alta varianza esperada de nuestro sistema estocástico con γ alto, nos indica que para obtener una estimación precisa, se requieran muchos más episodios que los 370 estimados previamente.

3. Argumenten formalmente por qué Monte Carlo no puede aplicarse directamente a tareas continuas.

Propongan una modificación concreta al algoritmo que permita aproximar Monte Carlo en una tarea continua y discutan las implicaciones de esa modificación sobre las garantías de convergencia.

Monte Carlo estima $V^\pi(s)$ a partir del retorno completo de un episodio $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$

En esta definición, literalmente tomamos en cuenta que $T < \infty$, es decir, que los episodios sean finitos para poder actualizarse. En una tarea continua donde nunca se termina el episodio, se rompe totalmente montecarlo.

Se rompe, precisamente por dos razones: 1, no esta garantizado que el retorno sea finito. 2 Nunca se llega al punto de actualizacion, donde MC realiza su calculo en reversa.

Una modificacion concreta vista en la literatura, es un truncamiento de n-pasos. En lugar de esperar el retorno completo de G_t , el cual puede diverger, se trunca la suma y se realiza una estimacion del valor actual, en lugar de hacerlo a partir de recompensas reales.

Esto tendria implicaciones en las garantías de convergencia, pues la recompensa ya no es un estimador insesgado de V^π , pues tomamos la estimacion actual del estado la cual es distinta del valor verdadero. Esto introduce un sesgo nuevo que es dependiente de que tan buena sea la estimacion actual. Esto hace que errores tempranos en V se propagen hacia atrás a travez del bootstrapping. Esto, sin embargo, se realiza a cambio de una reduccion de varianza, dado que reducimos la cantidad de pasos aleatorios en G_t .

Pero, hay que tener en cuenta que esta varianza reducida no es del todo buena, pues no es una varianza con respecto al valor real. Esta variacion del MC no nos garantiza que convergamos, y la ley de grandes numeros ya no nos ayuda dada la introduccion de este nuevo sesgo dependiente.

Task 3 (Entrega Final)

Implementen en Python **First-Visit** o **Every-Visit Monte Carlo Control** para el MDP diseñado en la Tarea 1.

La implementación debe incluir:

- Representación explícita del entorno como clase con métodos:

- `reset`
- `step`
- `render`

No usar librerías de RL.

- Implementación de Monte Carlo Control con la variante y estrategia de exploración elegidas en la Tarea 1, con soporte para registrar:
 - El número de episodios hasta convergencia.
 - La evolución de $Q(s, a)$ durante el entrenamiento.
- **Criterio de convergencia** explícito y justificado.

Definan cuándo consideran que Q ha convergido y por qué ese criterio es apropiado para Monte Carlo.

- Visualización del mapa con los valores

$$V(s) = \max_a Q(s, a)$$

codificados en color y la política greedy superpuesta como flechas direccionales, para al menos tres puntos del entrenamiento:

- Inicio.
 - Mitad.
 - Convergencia.
- Comparación directa con **Value Iteration**.

Ejecuten Value Iteration sobre el mismo MDP y comparen las políticas óptimas resultantes.

Si difieren en algún estado, argumenten por qué.

Task 4 (Entrega Final)

Con base en los resultados de la implementación, realicen:

1. Verificación de hipótesis

Contrasten cada hipótesis formulada en la Tarea 1 con los resultados observados.

Para cada hipótesis indiquen si fue:

- Confirmada.
- Refutada.
- Inconclusa.

Expliquen por qué.

2. Análisis de convergencia

Grafiquen la evolución de

$$\|Q_k - Q^*\|_\infty$$

en función del número de episodios, donde Q^* se aproxima con la solución de Value Iteration.

Respondan:

- ¿La convergencia es monótona?
- ¿Qué factores del diseño del MDP o del algoritmo afectan la velocidad de convergencia?

3. Sensibilidad a ϵ

Repitan el experimento con al menos tres valores distintos de ϵ .

Grafiquen el retorno promedio por episodio en función del número de episodios para cada valor.

Respondan:

- ¿Existe un valor de ϵ que domine a los demás en este dominio?
- ¿Ese resultado generaliza o depende del MDP específico?

4. Investigación bibliográfica

Busquen y lean un paper publicado entre **2020 y 2025** que aplique métodos Monte Carlo o sus extensiones a un problema real.

El paper debe provenir de una fuente indexada como:

- NeurIPS
- ICML
- ICLR
- JMLR
- o similar.

Escriban un resumen técnico de media página que incluya:

- El problema que resuelve.
- Cómo aplica o extiende Monte Carlo.
- Los resultados principales.
- Una reflexión sobre qué limitaciones del Monte Carlo clásico discutidas esta semana resuelve o no resuelve ese trabajo.

5. Dictamen técnico

Redacten un párrafo dirigido al equipo directivo de la empresa de videojuegos argumentando:

- Si Monte Carlo es una solución viable para generar comportamiento de personajes en el dominio descrito.
- Cuáles son sus limitaciones principales en ese contexto específico.
- Qué metodología recomendarían como paso siguiente considerando lo que saben sobre **Temporal-Difference Learning**.

Entregas en Canvas

1. Documento PDF con las respuestas a cada task.
 2. En la entrega parcial se espera que entreguen lo señalado. En la entrega final deben entregar **todos los tasks**.
 3. Archivo `.ipynb` o enlace a un repositorio de GitHub.
 - No se aceptan entregas por otros medios.
 - El código debe estar comentado explicando la relación con las fórmulas de las diapositivas.
-

Evaluación

Task	Puntaje
Task 1	0.60 pt
Task 2	0.60 pt
Task 3	0.90 pt
Task 4	0.90 pt
Total	3.0 pts